BÖLÜM 10 / MODULE 10

Kontrol Listeleri, Yaygın Hatalar ve MAANG Mimar Seviyesi Ustalık

Checklists, Common Mistakes & MAANG Architect Mastery

24 maddelik şartname kontrol listesi, 10 yaygın hata (5 klasik + 5 AI çağı tuzağı), monitör kenarı adım yazma kartı (Desk Card), MAANG sistem tasarımı mülakatlarında AIUP, otonom çoklu ajan (Multi-Agent) iş akışları ve Staff+ Seviye Gereksinim Mimarlığı yönetişimi.

The 24-point specification checklist, the ten common mistakes (5 classical + 5 AI-era traps), the split-pane step-writing desk card, AIUP in MAANG system design interviews, autonomous multi-agent code generation workflows, and Staff+ Requirements Architect governance.

55

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

24 Maddelik Şartname Kontrol Listesi: Reviewed ve Approved Eşiği

The 24-Point Specification Checklist: Gatekeeper for Reviewed & Approved States

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

24 MADDELİK ŞARTNAME KONTROL LİSTESİ TABLOSU

KATEGORİ	MADDELER
1. Kimlik ve Yapı (Identity & Structure) - Aktif fiil öbeği, kebab-case dosya adı, Tetikleyici vs Ön Koşul ayrımı	Maddeler 1 - 6 Sea-level hedefi, requirements.md bağı
2. Ön ve Son Koşullar (Pre & Postconditions) - Sistemce zorlanabilir ön koşul, asla akış içinde tekrar test edilmez	Maddeler 7 - 9 Başarı ve Hata minimum garantileri dolu!
3. Ana Başarı Senaryosu (Main Flow) - 3-9 adım, 'Top Kimde' ilk kelime, UI mekaniği (widget) temizlenmiş!	Maddeler 10 - 14 'Validates' kullanımı, 3 tür adımın dengesi
4. Alternatif Akışlar (Alternative Flows) - Her validasyona 1 akış, adıma çıpalı, kesin bitiş ('continues' / 'ends')	Maddeler 15 - 18 Açık uçlu akış YASAK! Alt hatalar birleştirildi
5. İş Kuralları ve Referanslar - BR-XXX eşleşmesi, entity_model bağı,	Maddeler 19 - 21 NFR ve Mockup linkleri
6. Üç Okuyucu Testi (The Three-Reader Test) - Zone 1 somut veriler, Kara Liste sıfır! Paydaş onaylayabilir mi?	Maddeler 22 - 24 Seneye stack değişse bu cümle doğru kalır mı?

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir uzay mekiği fırlatılmadan önce astronotların ve mühendislerin elindeki kontrol listesini (checklist) düşünün: 'Oksijen vanası açık mı? Basınç tamam mı? Yakıt kapağı kilitli mi?' Tek bir madde bile işaretlenmezse roket fırlatılmaz! 24 maddelik use case kontrol listesi, kod üretim butonuna basmadan önceki o fırlatma kontrol listesidir!

🇬🇧 ENGLISH (ELI5)

Think of the pre-launch checklist used by NASA astronauts and flight controllers before a space shuttle liftoff: 'Oxygen valves open? Thruster pressure nominal? Fuel hatch sealed?' If a single checkbox is missed, the launch is aborted! The 24-point specification checklist is the mandatory pre-flight audit before triggering AI code generation!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

24 Maddelik Şartname Kontrol Listesinin Mimarisi: Cockburn'ün klasik geçiş/kalış (pass/fail) testleri ile AI Unified Process'in katı doğrulama kurallarının birleşiminden oluşan bu liste, bir şartname 'Reviewed' ve ardından 'Approved' statüsüne geçmeden önce iki kez koşturulur. Liste 6 ana alana ayrılır: 1) Kimlik ve Yapı: İsim kullanıcının dilinde bir eylem öbeğidir, dosya adı `UC-XXX-\<kebab-name>.md` formatındadır, tek bir deniz seviyesi hedefi anlatır. 2) Ön ve Son Koşullar: Ön koşullar sistem tarafından zorlanabilir durumdadır ve akış içinde asla tekrar `if` ile test edilmez. Başarı ve hata son koşulları boşa çıkarılamaz bir sözleşmedir (no partial data). 3) Ana Akış: 3 ila 9 adımdır, her adımda topun sahibi ilk kelimedir, 'validates' kelimesi başarıyı mühürler, UI tıklama mekanikleri ayıklanmıştır. 4) Alternatif Akışlar: Her doğrulama adımının kesin bir hata akışı vardır; her akış bir adıma çıpalanır (`step 4`) ve mutlaka kesin bir dille sonlanır (`Use case continues at step N.` veya `Use case ends.`). 5) Kurallar ve Referanslar: Her BR kuralı en az bir adımdan referans alır; varlıklar `entity_model.md`'den gelir. 6) Üç Okuyucu Testi: Kara liste (JWT, SQL, bcrypt vb.) sıfır toleransla taranır; paydaş onayı ve 1 yıl sonraki teknoloji bağımsızlığı doğrulanır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 24-Point Specification Checklist Architecture: Merging Cockburn's classical pass/fail heuristics with the algorithmic verification rules of the AI Unified Process, this 24-point checklist gates specification promotion: executed once prior to 'Reviewed', and once more prior to 'Approved'. It spans six cohesive evaluation domains: 1) Identity & Structure: User-goal naming, strict kebab-case file paths matching `docs/use_cases.puml`, single-sitting sea-level scope, and requirements catalog linkages. 2) Boundary Guarantees: Preconditions must be enforceable assertions established upstream (never re-tested in-flow); failure postconditions must enforce transactional rollbacks (zero orphan state). 3) Scenario Discipline: 3-9 steps, active voice, ball-holder primacy, strict usage of 'validates', and zero UI widget micro-actions. 4) Deviation Robustness: 1-to-1 parity between validations and alternative flows, anchored triggers, and strictly banned open-ended terminations. 5) Referential Integrity: Business rules bi-directionally linked, domain nouns rooted in `docs/entity_model.md`. 6) Three-Reader Audit: Complete absence of Zone 3 blocklist terms, verifiable stakeholder outcomes, and enduring stack portability.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: 24 Maddelik Şartname Denetim Motoru (Spec Linter Pipeline)
package com.maang.aiup.checklist;

import java.util.List;

public class SpecChecklistAuditor {
    public record SpecCheckResult(int passedCount, int totalCount, List<String> violations) {
        public boolean isApproved() { return violations.isEmpty() && passedCount == totalCount; }
    }

    public static class AuditorEngine {
        public SpecCheckResult auditSpec(String specContent) {
            var violations = new java.util.ArrayList<String>();

            // Kontrol 1: Kara liste kelimeleri (Zone 3)
            List<String> blocklist = List.of("JWT", "SQL", "bcrypt", "SELECT", "INSERT", "SMTP");
            for (String forbidden : blocklist) {
                if (specContent.contains(forbidden)) {
                    violations.add("Kural 22 İhlali: Yasaklı Zone 3 kelimesi tespit edildi -> " +
forbidden);
                }
            }

            // Kontrol 2: 'checks whether' kullanımı (Kural 13)
            if (specContent.contains("checks whether")) {
                violations.add("Kural 13 İhlali: 'checks whether' yerine 'validates that'
kullanılmalıdır.");
            }

            int total = 24;
            int passed = total - violations.size();
            return new SpecCheckResult(passed, total, violations);
        }
    }

    void main() {
        var engine = new AuditorEngine();
        String sampleSpec = "Step 4: System validates that user account is active (BR-001).";
        var result = engine.auditSpec(sampleSpec);

        System.out.println("✅ Şartname Denetim Skoru: " + result.passedCount() + " / " +
result.totalCount());
        System.out.println("   Onay Durumu: " + (result.isApproved() ? "APPROVED" : "REJECTED"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir şartnameyi 'küçük bir hata var ama kod üretirken düzeltiriz' diyerek Approved statüsüne geçirmeyin! 24 kuralın tamamı yeşil olmadan kod üretim komutunu (/implement) çalıştırmayın.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never promote a spec to 'Approved' with known checklist violations under the assumption that AI will compensate. All 24 criteria must pass cleanly before generation.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• 24 maddelik liste şartnameyi Reviewed ve Approved aşamalarına taşıyan kapı bekçisidir.

• Liste kimlikten ön koşullara, senaryo ritminden kara liste denetimine 6 alanı kapsar.

• Tek bir ihlal bile AI'ın mimariyi yanlış üretmesine yol açabileceği için sıfır toleranslıdır.

🎯 Key Architectural Takeaways

• The 24-point checklist acts as the definitive gatekeeper for Reviewed and Approved states.

• It evaluates 6 domains: identity, pre/postconditions, scenario, branches, rules, and 3 readers.

• Zero tolerance: a single checklist defect invites prompt hallucination and layer misallocation.

56

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

En Yaygın 10 Hata: 5 Klasik Tuzak vs 5 AI Çağı Felaketi

The Ten Common Mistakes: 5 Classical Pitfalls vs 5 AI-Era Disasters

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                    EN YAYGIN 10 USE CASE HATASI VE TEDAVİSİ


🏛️ 5 KLASİK TUZAK (2001'den Beri Değişmeyenler):
1. Sistem Yok (No System): Sadece kullanıcı hareket eder, sistem hiçbir şey yapmaz!
2. Aktör Yok (No Actor): Sistemin içinden yazılmıştır; ekranda kullanıcı kaybolur!
3. UI Turu (UI Walkthrough): 6 adım boyunca tıklandı/yazıldı sayılır (En yaygın hata!).
4. Su Altı Seviye (Underwater Goal): 'ISBN Doğrula' gibi atomik işler use case sanılır!
5. Amaç-İçerik Uyuşmazlığı: 'Giriş Yap' içinde tüm sistemi tarif etmeye kalkmak!

🤖 5 AI ÇAĞI FELAKETİ (2026'nın Yeni Tehlikeleri):
6. Mekanizma Sızıntısı (Mechanism Leak): JWT, SQL, regex yazıp kodu kitlemek!
7. Gerçeğin Tekrarı (Restated Truth): Entity alanlarını şartnameye kopyalamak!
8. Açık Uçlu Akış (The Open Flow): Bitiş cümlesi olmayan sahipsiz alternatif dallar!
9. Açık Çek (The Blank Check): 'Uygun şekilde', 'ilgili veriler' gibi boş kelimeler!
10. Sonradan Yazılan Şartname (Backfilled Spec): Kodu yazıp şartnameyi sonra yamamak!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir pastacı düşünün: 5 tane klasik hata vardır (Fırını açmayı unutmak, unu döküp şekeri unutmak, yumurtayı kabuğuyla kırmak gibi). Bir de yeni nesil akıllı elektrikli fırınların getirdiği 5 hata vardır (Fırının wifi şifresini kek harcına karıştırmak, derece yerine fırının kodunu değiştirmek gibi). İşte bu 10 hata hem 20 yıldır yapılan hem de AI ile yeni başlayan en tehlikeli 10 tuzaktır!

🇬🇧 ENGLISH (ELI5)

Think of baking: There are 5 classical amateur blunders (forgetting to preheat, dumping uncracked eggs, confusing salt with sugar). And there are 5 brand-new modern digital hazards (spilling liquid on the microchip panel, overriding the firmware safety valve). The 10 common mistakes split evenly between eternal human blunders and fatal AI-era traps!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

10 Yaygın Hatanın Analizi ve Önleme Stratejileri: Martinelli'nin use case analizinde hatalar iki eşit kampa ayrılır: 1) Klasik Hatalar: Cockburn'ün 2001'de uyardığı tuzaklar bugün de devam etmektedir. 'Sistem Yok' hatasında kullanıcı her şeyi yapar ama sistem tepki vermez, sonuçta AI eylemsiz bir UI üretir. 'UI Walkthrough' hatasında 6 adımda form doldurma anlatılır, oysa 'Kullanıcı üye bilgilerini girer' denmelidir. 'Su Altı Hedef' hatasında tek bir buton kontrolü use case yapılır. 2) AI Çağı Hataları: 'Mekanizma Sızıntısı' (Zone 3 ihlali), şartnamenin teknoloji göçlerinde çöp olmasına yol açar. 'Gerçeğin Tekrarı' (Restated Truth), entity_model.md ile use case arasında çelişki yaratır ve AI sessizce uydurma yapar. 'Açık Uçlu Akış' (Open Flow), sonu belirtilmemiş bir alternatif daldır; insan bunu görmezden gelir ama AI kafasından bir bitiş uydurarak müşteriyi yanlış bir ekrana fırlatır. 'Açık Çek' (Blank Check), 'uygun şekilde işler' diyerek kararı AI'a bırakmaktır. 'Sonradan Yazılan Şartname' (Backfilled Spec) ise şartname tabanlı yazılımı tamamen öldüren en ölümcül günahtır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Analysis of the Ten Common Mistakes: Martinelli categorizes specification failures into five classical human biases and five novel AI-era failure modes: 1) Classical Pitfalls: 'No System' leaves out automated responses, yielding inert UI forms; 'No Actor' specifies internal batch routines, discarding presentation layers; 'UI Walkthrough' recites keystrokes rather than intent; 'Underwater Goals' elevate atomic validation routines to full use cases; 'Purpose-Content Mismatch' overloads auxiliary flows. 2) AI-Era Disasters: 'Mechanism Leakage' bakes ephemeral protocol jargon (JWT, bcrypt, SQL) into living assets; 'Restated Truth' creates duplicate schemas that drift and cause silent LLM reconciliation bugs; 'The Open Flow' omits explicit terminations, causing AI to hallucinate arbitrary navigation jumps; 'The Blank Check' uses lazy vocabulary ('handles appropriately', 'etc.') delegating core business decisions; 'Backfilled Specs' updates documentation retroactively, destroying spec-first authority.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: 10 Yaygın Hatanın Önlenmesi (Açık Çek ve Açık Uçlu Akış Tedavisi)
package com.maang.aiup.mistakes;

public class CommonMistakesRemedyDemo {
    // ❌ HATA 8: Açık Uçlu Akış (Ne olacağı belirsiz):
    // if (!valid) return; // Ne döndü? Log mu attı? Ekrana ne dedi?

    // ✅ TEDAVİ: Kesin Bitiş ve Minimum Garanti (Failure Postcondition)
    public sealed interface FlowResult permits FlowResult.Success, FlowResult.Halted {
        record Success(String orderId) implements FlowResult {}
        record Halted(String errorCode, String userMessage) implements FlowResult {}
    }

    public static class SafeOrderHandler {
        public static FlowResult execute(int stock) {
            if (stock <= 0) {
                // Alternatif Akış A1: Açık uçlu değil! Kesin bitiş ve mesaj var:
                return new FlowResult.Halted("ERR-01", "A1: Stok tükenmiştir. Use case sonlanır.");
            }
            return new FlowResult.Success("ORD-999");
        }
    }

    void main() {
        var result = SafeOrderHandler.execute(0);
        switch (result) {
            case FlowResult.Success s -> System.out.println("Başarı: " + s.orderId());
            case FlowResult.Halted h -> System.out.println("✅ Hata 8 ve 9 Önlendi: " +
h.userMessage());
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamenizde 'vb.', 've benzeri', 'uygun şekilde işler' kelimelerini gördüğünüz anda silin! Her biri AI'a kafasına göre uydurma yetkisi veren birer açık çektir.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Search your specification for 'etc.', 'and so on', or 'appropriately'. Erase them immediately. Every vague qualifier delegates architectural control to LLM randomness.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- 5 klasik hata (UI walkthrough, su altı hedefi) 20 yıldır şartnamelerin en büyük derdidir.
- 5 AI çağı hatası (mekanizma sızıntısı, açık uçlu akış, açık çek) üretimi doğrudan zehirler.
- Sonradan şartname yazmak (backfilling) şartname tabanlı kalkınmanın ölüm fermanıdır.

🎯 Key Architectural Takeaways

- 5 classical blunders (UI walkthrough, underwater goals) remain persistent pitfalls.
- 5 AI-era disasters (mechanism leaks, open flows, blank checks) directly corrupt code generation.
- Backfilling specifications after coding completely destroys spec-driven development.

57

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

Masaüstü Adım Yazma Kartı (Step-Writing Desk Card): Bölünmüş Ekranda Hızlı Rehber

The Step-Writing Desk Card: The Split-Pane Reference for Engineers

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              MASAÜSTÜ ADIM YAZMA KARTI (STEP-WRITING CARD)              │
└──────────────────────────────────────────────────────────┘

 Mühendisler İçin Monitörün Sağında Açık Tutulacak Özet Kart:

 ┌──────────────────────────────────────────────────────────┐
 │ 📝 BİR ADIM YAZARKEN (FOR MAIN SCENARIO STEPS):          │
 │ 1. Tek bir cümle. Geniş / Şimdiki zaman. Etken çatı (Active Voice).  │
 │ 2. İLK KELİME = Top kimdeyse o aktörün adı!              │
 │ 3. Formül: Özne → Yüklem → Nesne → Edat Öbeği.           │
 │ 4. Hareketleri değil niyeti yaz; tek yöndeki verileri tek adımda topla! │
 │ 5. ASLA "checks whether" deme; "validates that..." de!   │
 │ 6. Gözlemlenebilir alanda SOMUT ol (Kolon adları, net sayılar, mesaj). │
 │ 7. Mekanizmada TAMAMEN SESSİZ kal (Protokol, şema, DB, kod adı YOK!). │
 │ 8. 3 ila 9 adım arası yaz. Son adım = Hedefin teslim edildiği andır! │
 ├──────────────────────────────────────────────────────────┤
 │ 🔀 ALTERNATİF AKIŞ YAZARKEN (FOR ALTERNATIVE FLOWS):    │
 │ 1. Başlık: ### A#: <Durumun Adı>                         │
 │ 2. Tetikleyici: **Trigger:** <Sistemin yakaladığı durum> (step N) │
 │ 3. 1..N Adım: Yukarıdaki adım kurallarına harfiyen uyan adımlar. │
 │ 4. Kesin Bitiş: "Use case continues at step N." VEYA "Use case ends." │
 │ 5. "ends" ile biten her akış için Failure Postcondition garantisi şart! │
 └──────────────────────────────────────────────────────────┘
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Uçak pilotlarının kokpitindeki küçük acil durum kartlarını düşünün: Uçakta motor arızası çıktığında pilot 500 sayfalık kalın kitabı açıp okumaz; hemen direksiyonun yanındaki 8 maddelik küçük karta bakar! Adım yazma kartı, şartname yazarken monitörünüzün sağ köşesine yapıştıracağınız o hayat kurtaran kokpit kartıdır!

🇬🇧 ENGLISH (ELI5)

Think of the quick-reference checklist clipped directly to an airline pilot's yoke: In severe turbulence, pilots don't flip through a 600-page manual; they scan the laminated 8-point card right in front of them! The Step-Writing Desk Card is that vital cockpit card, pinned to your split-pane IDE window while drafting specs!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Masaüstü Adım Yazma Kartının Pratik Değeri: Simon Martinelli kitabın 10.3 bölümünde bu kartı özel olarak tek sayfalık bir referans olarak sunar. Çünkü kimse şartname yazarken 60 sayfalık metodoloji kitabını baştan sona okumaz; mühendisler ikiye bölünmüş monitörde (split-pane) bir tarafta use case yazarken diğer tarafta hızlı kurallara bakmak ister. Bu kart iki kompakt bloktan oluşur: 1) Senaryo Adımları: 8 altın kural. Özne-yüklem uyumu, topun sahibinin ilk kelime olması, 'validates that' kalıbı, Zone 1 somutluğu ve Zone 3 sessizliği. 2) Alternatif Akışlar: 5 altın kural. Adıma çıpalama (`step N`), 1..N adımlı işleme ve mutlaka iki yasal bitişten birisi (`continues at step N` veya `ends`). Bu kartı takip eden bir mühendisin şartnamesi `/spec-review` linter'ından ilk seferde %100 geçer not alır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Practical Utility of the Step-Writing Desk Card: In Section 10.3, Martinelli crystallizes the entire writing discipline into a desk card designed specifically for split-pane engineering environments. Software engineers and business analysts drafting specifications in an IDE do not consult a full-length book during authoring; they need an ambient, unencumbered reference visible on their secondary monitor or split editor. The card establishes two airtight syntactic blocks: 1) Scenario Step Rules: 8 constraints enforcing active voice, ball-holder primacy, intent encapsulation, strict 'validates that' grammar, Zone 1 observability, Zone 3 silence, and 3-9 step bounds. 2) Alternative Flow Rules: 5 constraints governing condition titling, anchored triggers (`step N`), and explicit binary exits. Adhering to the desk card guarantees zero lint violations during `/spec-review` passes.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Masaüstü Kartı Kurallarına %100 Uyumlu Temiz Use Case Modeli
package com.maang.aiup.deskcard;

public class DeskCardComplianceDemo {
    // Kural 1-3: Aktör net (Üye, Sistem). Kural 5: Validates kuralı.
    public record BorrowContext(String memberId, String bookId, boolean isAvailable) {}

    public static class StepExecutionEngine {
        // Adım 4: Sistem kopyanın mevcut olduğunu doğrular (BR-004)
        public static String step4ValidateAvailability(BorrowContext ctx) {
            if (!ctx.isAvailable()) {
                // A1 Akışı: Tetikleyici step 4'e bağlı. Bitiş: Use case ends.
                return "A1: Kopya mevcut değil. En erken iade tarihi gösterildi. Use case ends.";
            }
            // Başarı yolu:
            return "Adım 4 Başarılı: Kopya mevcut. Use case continues at step 5.";
        }
    }

    void main() {
        var availableCtx = new BorrowContext("M1", "BK-9", true);
        var unavailableCtx = new BorrowContext("M1", "BK-9", false);

        System.out.println(StepExecutionEngine.step4ValidateAvailability(availableCtx));
        System.out.println(StepExecutionEngine.step4ValidateAvailability(unavailableCtx));
        System.out.println("✅ Adım Yazma Kartı kuralları koda kusursuz yansıdı.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Desk kartındaki formülü bozup 'Kitap mevcut ise sisteme kaydet' şeklinde şartlı (if-then) cümleler kurmayın! Şartları daima alternatif akışlara taşıyın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never violate the desk card syntax by embedding conditional 'if-then' clauses within a step. Main steps assert success; deviations belong strictly in alternative flows.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Masaüstü kartı, çift ekranlı çalışırken split-pane'de açık tutulacak en pratik kaynaktır.
- Adım kuralları (özne başta, validates, 3-9 adım) hatasız bir ana akış kurar.
- Akış kuralları (adıma çıpalama, net bitiş) açık uçlu hataları tamamen engeller.

🎯 Key Architectural Takeaways

- The desk card is the definitive split-pane reference for daily specification authoring.
- Step rules (ball-holder first, validates, 3-9 steps) construct an unbranching happy path.
- Flow rules (anchored triggers, explicit exits) eliminate dangling failure paths entirely.

58

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

MAANG Sistem Tasarımı Mülakatı: Yüksek Seviye Mimariden AIUP Şartnamesine

MAANG System Design Interview: Translating High-Level Architecture into AIUP Specs

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                MAANG SYSTEM DESIGN MÜLAKATINDA AIUP DÖNÜŞÜMÜ                │
└──────────────────────────────────────────────────────────────┘

MÜLAKAT SORUSU (Meta / Google / Amazon):
"Bize ölçeklenebilir bir Twitter / X veya Uber rezervasyon sistemi tasarlayın."

GELENEKSEL ADAY (Bocalayan):
- Hemen beyaz tahtaya 'Redis', 'Kafka', 'Cassandra' kutuları çizer.
- Hangi aktörlerin ne yaptığını, hangi iş kurallarının olduğunu unutur.
- Mülakatçı sorar: "Peki bakiye yetersizse ne olacak?" Aday tıkanır!

🌟 MAANG STAFF+ ADAY (AIUP ile Kazanan):
1. ADIM: Aktörleri ve Deniz Seviyesi Hedefleri Belirler (UC-001, UC-002).
2. ADIM: Kapsamı In/Out Tablosu ile Netleştirir (Scope Boundary).
3. ADIM: 3-9 Adımlı Ana Senaryoyu ve Alternatif Akışları (A1, A2) Yazar.
4. ADIM: Mekanizmayı (Kafka, Redis) Zone 3 olarak şartnameden soyutlar;
         mimaride bu bileşenleri iş kurallarının (BR) altına birer Adapter olarak bağlar!

🎯 MÜLAKATÇININ YORUMU:
"Bu aday sadece kutu çizmeyi değil; bir sistemi yaşayan, test edilebilir ve
 otonom ekiplerce üretilebilir bir şartname zincirine oturtmayı biliyor!"
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir inşaat mühendisliği sınavını düşünün: Acemi aday hemen 'Ben 50 tonluk çimento mikseri getireceğim' diye aletleri anlatmaya başlar. Usta mimar ise önce binada kimlerin yaşayacağını, kapıların nereye açılacağını, deprem anında nereye kaçılacağını (Use Case) çizer! Mikser zaten çağrılır. MAANG mülakatında önce hayatı ve kuralları çizmek sizi birinci yapar!

🇬🇧 ENGLISH (ELI5)

Think of an architectural board exam: An amateur candidate immediately brags: 'I will order 50 tons of hydraulic concrete mixers!' The master architect first presents the residential floor plan, primary fire escape egress routes, and earthquake load specs (Use Cases). The concrete mixer is just plumbing. In a MAANG interview, framing user-goal invariants first earns the Staff+ offer!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

MAANG Sistem Tasarımı Mülakatlarında AIUP Yaklaşımı: Meta, Amazon, Apple, Netflix ve Google (MAANG) mülakatlarında Staff ve Principal seviyesindeki adaylardan sadece dağınık kutular ve 'Redis cache koyalım' demeleri beklenmez. Mülakatçıların aradığı en kritik yetkinlik 'Sistemik Netlik' (Systemic Clarity) ve 'Sınırları Doğru Çizebilme' (Bounded Contexts) kabiliyetidir. AIUP disiplini adaya şu muazzam avantajları sağlar: 1) Gereksinim Netleştirme: Mülakatın ilk 5 dakikasında aktörleri ve deniz seviyesi hedefleri belirleyip bir In/Out tablosu çizen aday, kontrolü eline alır. 2) Senaryo ve Hata Hakimiyeti: Ana akışı 'validates' kalıbıyla kurup alternatif akışları (stok yetersiz, ödeme zaman aşımı) numaralı adımlara bağlayan aday, mülakatçının en sevdiği 'Peki ya sistem çökerse?' sorularının tamamını önceden yanıtlamış olur. 3) Temiz Mimari Ayrımı: Mekanizmayı (Zone 3: Kafka, DynamoDB) iş kurallarından (Zone 1) ayrı bir katmanda sunarak, sistemin teknoloji bağımsızlığını ve test edilebilirliğini kanıtlar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

AIUP Strategy in MAANG System Design Interviews: In Staff- and Principal-level design loops at Google, Meta, or Amazon, whiteboarding arbitrary distributed components (Kafka, Cassandra, Redis) without behavioral boundaries signals architectural immaturity. Interviewers assess an architect's ability to tame ambiguity, define bounded contexts, and enforce operational resilience. Applying the AI Unified Process establishes immediate command of the interview: 1) Scoping with In/Out Lists: The candidate partitions the problem space within 5 minutes, eliminating scope creep. 2) Explicit Behavioral Workflows: Presenting sea-level use cases where each validation is paired with an alternative failure flow demonstrates failure-domain mastery. 3) Strict Separation of Zones: By cleanly isolating business rules (BR-XXX) and observable outcomes (Zone 1) from underlying infrastructure adapters (Zone 3: distributed cache, write-ahead log), the candidate demonstrates clean architectural rigor.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: MAANG Seviyesi Ölçeklenebilir Sistem Tasarımı Use Case Mimarisi
package com.maang.aiup.maangdesign;

import java.util.concurrent.CompletableFuture;

public class MaangSystemDesignDemo {
    // 1. USE CASE MODELİ (Teknoloji bağımsız iş mantığı)
    public record RideRequest(String riderId, double pickupLat, double pickupLon) {}
    public record RideMatched(String rideId, String driverId, double estimatedFare) {}

    public interface DriverMatchingPort {
        CompletableFuture<String> matchNearestDriver(double lat, double lon);
    }

    public static class RequestRideUseCase {
        private final DriverMatchingPort matchingPort;

        public RequestRideUseCase(DriverMatchingPort matchingPort) {
            this.matchingPort = matchingPort;
        }

        public CompletableFuture<RideMatched> requestRide(RideRequest request) {
            // Şartname Kuralı: En yakın sürücüyü bul, eşleşirse başarı döndür
            return matchingPort.matchNearestDriver(request.pickupLat(), request.pickupLon())
                .thenApply(driverId -> new RideMatched("RIDE-101", driverId, 45.50));
        }
    }

    void main() {
        DriverMatchingPort mockKafkaPort = (lat, lon) -> CompletableFuture.completedFuture("DRIVER-
ALICE");
        var useCase = new RequestRideUseCase(mockKafkaPort);

        useCase.requestRide(new RideRequest("RIDER-1", 41.0, 29.0))
            .thenAccept(res -> System.out.println("✅ MAANG Mülakat Mimarisi Çalıştı: " + res));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Sistem tasarımı mülakatında hemen veritabanı şeması veya Kafka topic'i çizmeye başlamayın; önce aktörleri, hedefleri ve In/Out kapsamını netleştirin.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not dive straight into Kafka partitions or database schemas during a system design interview. Pin down actors, user goals, and the In/Out boundary first.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• MAANG mülakatlarında kutu çizmeden önce aktörler, hedefler ve In/Out listesi sunulur.

• Ana akış ve alternatif akış disiplini, mülakatçının 'Peki ya hata olursa?' sorularını peşinen çözer.

• Zone 1 iş mantığını Zone 3 altyapısından ayırmak adayı Staff+ mimar seviyesine taşır.

🎯 Key Architectural Takeaways

• Present actors, user goals, and an In/Out boundary before drawing architecture boxes.

• Scenario and alternative flow discipline preemptively resolves failure-mode questions.

• Decoupling Zone 1 business invariants from Zone 3 plumbing signals Staff+ design maturity.

59

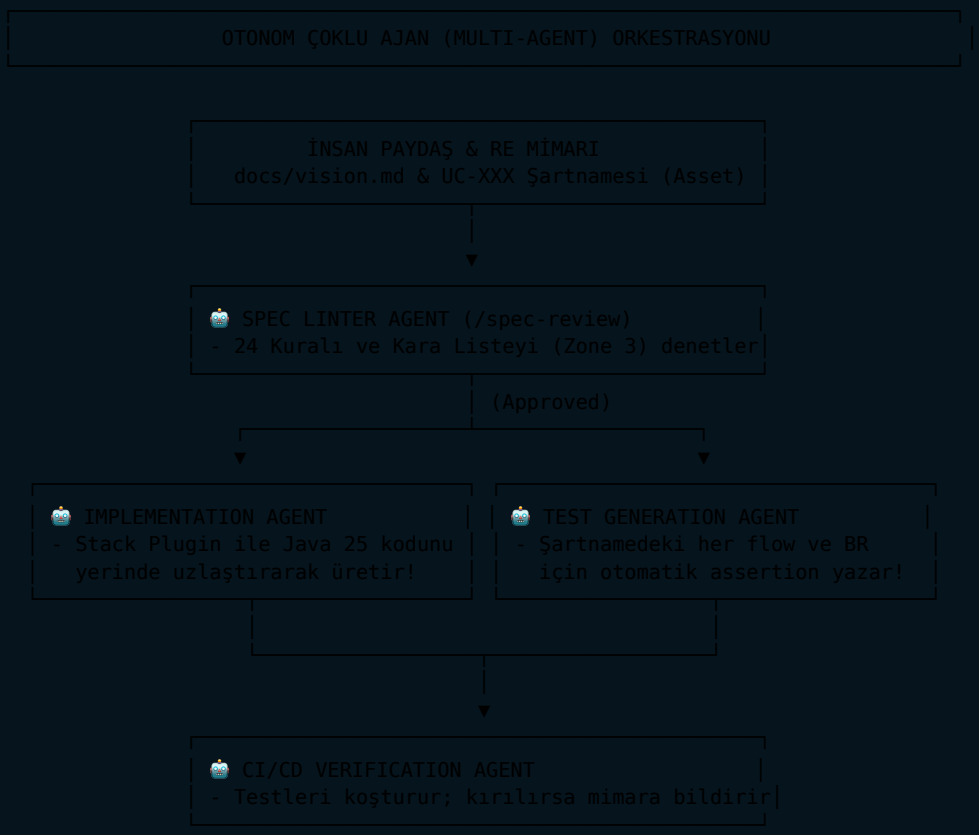
BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

Otonom Çoklu Ajan (Multi-Agent) İş Akışları ve Şartname Tabanlı Kod Üretimi

Autonomous Multi-Agent AI Workflows: Orchestrating Spec-Driven Code Generation

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir inşaat şantiyesindeki robot işçileri düşünün: Mimar (İnsan) elindeki kesin mimari planı (Use Case Şartnamesi) robot şefe verir. Robot şef planı kontrol eder. Sonra bir robot duvarları örerken (Kod Ajanı), diğer robot betonun sağlamlığını lazerle ölçer (Test Ajanı). Kimse kafasına göre bina yapmaz; herkes mimarın tek bir çizimine bakar!

🇬🇧 ENGLISH (ELI5)

Imagine a high-tech robotic construction site: The human architect provides the structural CAD blueprint (The Use Case Spec). A supervisor robot audits the blueprint. Then, while one robotic arm lays the precision steel beams (Coding Agent), a companion drone fires lasers to verify structural integrity (Testing Agent). All autonomous agents align around a single blueprint!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Otonom Çoklu Ajan Orkestrasyonu (Multi-Agent Workflows): Yapay zekâ yazılım mühendisliği artık tek bir 'Chat GPT' penceresine prompt yazmaktan çıkmış, uzmanlaşmış çoklu ajanların (Multi-Agent Systems) işbirliğine dönüşmüştür. Ancak kontrolsüz ajanlar birbirlerinin kodunu bozabilir ve halüsinasyon sarmalına girebilir. AI Unified Process bu sorunu şartnameyi 'ortak koordinasyon düzlemi' (shared coordination substrate) yaparak çözer: 1) Spec Linter Ajanı: Şartnamenin sözdizimini, 24 kuralını ve Zone 3 temizliğini denetler. 2) Implementasyon Ajanı: Şartnameden sadece onaylanmış adımları okuyarak Java kodunu üretir. 3) Test Üretim Ajanı: Doğrulama adımlarını ve BR kurallarını bağımsız olarak test koduna dönüştürür. Kod üreten ajan ile test üreten ajanın birbirlerinden habersiz olarak sadece ortak şartnameyi okuması, 'kendi yazdığı kodu test etme' önyargısını tamamen ortadan kaldırır. 4) Doğrulama Ajanı: Kod ve testleri derleyip CI/CD ortamında koşturur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Orchestrating Spec-Driven Multi-Agent Workflows: Autonomous AI software development has outgrown monolithic chat interfaces in favor of specialized multi-agent architectures. Without rigorous guardrails, multi-agent pipelines succumb to hallucination feedback loops and code drift. The AI Unified Process leverages the specification artifact chain as the shared immutable state substrate across agent topologies: 1) Spec Review Agent: Executes deterministic static analysis and advisory reviews on `UC-XXX.md` artifacts. 2) Code Implementation Agent: Ingests approved specs and stack plugins to synthesize domain services and presentation controllers. 3) Test Generation Agent: Reads failure postconditions and business rules to synthesize orthogonal test suites. Because the test agent works independently from the specification rather than inspecting generated production code, it prevents the classic bias of AI tests confirming AI mistakes. 4) Verification Agent: Runs test suites within sandboxed CI environments, providing deterministic feedback loops.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Çoklu Ajan Koordinasyon Arayüzü (Agent Orchestration Substrate)
package com.maang.aiup.multiagent;

public class MultiAgentOrchestratorDemo {
    public record SpecArtifact(String id, String content, boolean isApproved) {}
    public record GeneratedCode(String sourceFile, String testFile) {}

    // Ajan 1: Şartname Denetleyicisi
    public static class SpecLinterAgent {
        public boolean verify(SpecArtifact spec) {
            return spec.isApproved() && !spec.content().contains("SQL");
        }
    }

    // Ajan 2: Kod Üreticisi
    public static class CodeGeneratorAgent {
        public GeneratedCode generate(SpecArtifact spec) {
            return new GeneratedCode("LoanService.java", "LoanServiceTest.java");
        }
    }

    void main() {
        var spec = new SpecArtifact("UC-002", "System validates that copy is available (BR-004).",
true);
        var linter = new SpecLinterAgent();
        var generator = new CodeGeneratorAgent();

        if (linter.verify(spec)) {
            var code = generator.generate(spec);
            System.out.println("🤖 Çoklu Ajan İş Akışı Başarılı!");
            System.out.println("   -> Üretilen Kod: " + code.sourceFile());
            System.out.println("   -> Üretilen Bağımsız Test: " + code.testFile());
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Kod üreten ajan ile test üreten ajanı aynı prompt zincirinde çalıştırmayın! Test ajanı koda bakarak değil, sadece şartnameye bakarak test yazmalıdır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never permit the test-generation agent to inspect generated production source code. Tests must be synthesized strictly from the specification to prevent self-affirming hallucinations.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Şartname, çoklu yapay zekâ ajanlarının etrafında toplandığı tek ortak gerçektir.

• Kod üreten ajan ile test üreten ajan birbirinden bağımsız çalışarak tarafsızlığı korur.

• Ajanlar arası iletişim serbest sohbetle değil, yapısal şartname zinciriyle yönetilir.

🎯 Key Architectural Takeaways

• The specification serves as the shared coordination substrate across autonomous AI agents.

• Code and test generation agents operate orthogonally to eliminate confirmation bias.

• Inter-agent workflows are governed by the structured artifact chain, not conversational chat.

60

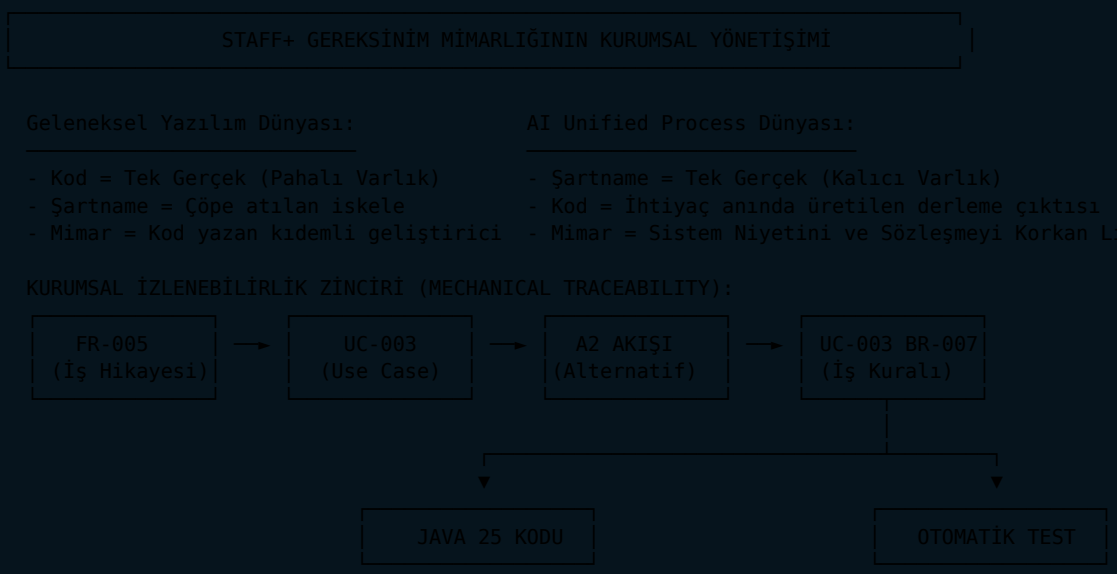
BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

Staff+ Seviye Gereksinim Mimarlığı: Yönetişim, İzlenebilirlik ve AIUP Çağı

The Staff+ Requirements Architect: Governance, Traceability & The AIUP Future

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Dünyanın en büyük orkestrasını düşünün: 100 tane kemancı, 50 tane trompetçi vardır. Trompetçi değişebilir, kemancı emekli olabilir, hatta enstrümanlar ahşaptan elektroniğe geçebilir. Ama Mozart'ın yazdığı nota kâğıdı (Partisyon / Şartname) 300 yıldır aynı ihtişamla çalınır! Staff+ Mimar, o ölümsüz partisyonu yazan ve tüm orkestrayı yöneten baş şeftir!

🇬🇧 ENGLISH (ELI5)

Think of the world's greatest philharmonic orchestra: 100 violinists, 50 brass players. Musicians retire, violinists are replaced, instruments may transition from acoustic to carbon-fiber. But Mozart's symphonic score (The Specification) endures across centuries with flawless fidelity! The Staff+ Architect is the master conductor who preserves the eternal score.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Staff+ Seviye Gereksinim Mimarlığı ve Kurumsal Yönetişim: Yazılım dünyasında 40 yıl boyunca 'Kod tek gerçektir' yalanıyla yaşadık. Bu yaklaşım, sistemlerin 5 yılda bir teknik borç bataklığına saplanmasına ve milyonlarca dolarlık 'legacy rewrite' facialarına yol açtı. AI Unified Process ve Simon Martinelli'nin metodolojisi yazılım mühendisliğini gerçek bir mühendislik disiplinine kavuşturur: Kod artık derlenip atılabilen geçici bir ikincil çıktıdır; asıl şirket varlığı (asset) ise gereksinimlerden use case'lere, oradan entity model ve testlere uzanan 'Şartname Zinciri'dir (Artifact Chain). Staff+ Requirements Architect; bu zincirin mekanik izlenebilirliğini (Traceability: `FR -> UC -> Flow -> BR -> Code -> Test`), kara liste kontrollerini ve alan modellerini yöneten kurumsal stratejisttir. Bir regülatör veya müşteri 'Neden bu işlem reddedildi?' diye sorduğunda, kod aramak yerine saniyeler içinde kural kimliğini gösterir. Şirketin teknoloji yığını yarın Spring Boot'tan başka bir platforma geçse dahi, şartname zinciri sayesinde tek bir iş kaybı olmadan tüm sistem saatler içinde yeniden üretilebilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Staff+ Requirements Architecture and Enterprise Governance: For four decades, the software industry operated under the economic premise that 'the code is the truth'—an assumption that produced unmaintainable monoliths, trillion-dollar technical debt, and recurrent failed re-writes. The AI Unified Process permanently inverts this paradigm. Code is demoted to an ephemeral, regenerable compilation artifact. The durable enterprise asset is the Specification Artifact Chain: Vision -> Requirements Catalog -> Entity Model -> Use Case Diagram -> Use Case Specifications -> Test Journeys. The Staff+ Requirements Architect is the strategic custodian of this ecosystem, enforcing mechanical traceability (`FR-XXX -> UC-XXX -> Flow -> BR-XXX -> Code -> Test`), lexical hygiene, and architectural boundaries. When auditors or clients query why a transaction was declined, the answer is a deterministic chain of identifiers rather than code archaeology. When infrastructure stacks evolve, the enterprise regenerates its entire portfolio with zero business friction.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Staff+ Seviye Mekanik İzlenebilirlik Zinciri (Traceability Metadata)
package com.maang.aiup.staffarchitect;

import java.lang.annotation.*;

public class EnterpriseTraceabilityDemo {
    // Kurumsal İzlenebilirlik Notasyonu
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.METHOD)
    public @interface Traceability {
        String requirementId();
        String useCaseId();
        String flowId();
        String businessRule();
    }

    public static class EnterpriseLoanEngine {
        @Traceability(
            requirementId = "FR-012",
            useCaseId = "UC-002",
            flowId = "A2",
            businessRule = "BR-005: Loan Limit"
        )
        public boolean evaluateLoanQuota(int currentLoans) {
            // Mekanik zincir: Koddaki her satır doğrudan iş kuralına bağlanır!
            return currentLoans < 5;
        }
    }

    void main() throws NoSuchMethodException {
        var method = EnterpriseLoanEngine.class.getMethod("evaluateLoanQuota", int.class);
        var trace = method.getAnnotation(Traceability.class);

        System.out.println("🏛️ Staff+ Kurumsal İzlenebilirlik Raporu:");
        System.out.println("   -> İlgili Gereksinim: " + trace.requirementId());
        System.out.println("   -> İlgili Use Case  : " + trace.useCaseId());
        System.out.println("   -> İlgili Akış      : " + trace.flowId());
        System.out.println("   -> Uygulanan Kural  : " + trace.businessRule());
        System.out.println("✅ Kod doğrudan iş şartnamesine bağlandı. AIUP Tamamlandı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartname zincirini sadece dokümantasyon olarak görmeyin; o zincir şirketin yazılımsal DNA'sıdır. Kod ölür, şartname yaşar!

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never treat the artifact chain as mere bureaucratic documentation. The specification is the living genetic code of the enterprise. Code is ephemeral; specifications endure.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• AI Unified Process ile yazılım mühendisliği gerçek bir mühendislik disiplinine kavuşur.

• Mekanik izlenebilirlik (FR -> UC -> Flow -> BR -> Code -> Test) kurumsal şeffaflığı sağlar.

• Şartname tek kalıcı varlıktır; kod yarın başka bir dile veya stack'e saniyeler içinde taşınabilir.

🎯 Key Architectural Takeaways

• The AI Unified Process elevates software development into a true engineering discipline.

• Mechanical traceability connects every line of code to a specific business requirement.

• Specifications are the sole durable asset; code can be regenerated onto new stacks effortlessly.